

DD: OSR-01 — Stock Chain Engine

Developer implementation guide: DD_OSR-01-Stock_Chain_Engine-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_OSR-01-Stock_Chain_Engine-v1.0.md.

Verdict: ■ New | **Wave:** 2 | **Group O:** Operations Support Resources | **Version:** v1.0 | **Module:** Stock Chain The system of record for **equipment stock balances + movements** across Wananchi's two-tier stock topology: central warehouse (one per operator) and contractor vans (one logical stock location per contractor). Every router, ONT, splitter, wall socket, and cable that enters or leaves any location creates a movement row; balances are derived projections. Owned at operator scope. References `equipment_sku.sku_id` from PLM-CFG-06 (this module does not define SKUs); references `contractor_id` from EM-02 (this module does not define contractors). v1.0 scope: **quantity tracking only at the SKU level — no individual equipment instance / serial number tracking**. Serialized instance tracking is deferred to a future DD (OSR-INSTANCE-01 or equivalent). Returns + RMA workflows in v1.0 work at the SKU + qty level.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/08_osr_equipment/DD_OSR-01-Stock_Chain_Engine-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/stock-reservations/{id}
- DELETE /api/stock-reservations/{reservation_id}
- GET /api/stock-availability/{wo_context}
- GET /api/stock-balances?location=X&sku=Y
- GET /api/stock-balances?operatorCode=WIK&locationId=WIK-VAN-contractor_ke_nairobi_a
- GET /api/stock-locations/{location_id}

- GET /api/stock-locations?operatorCode=WIK&active=true
- GET /api/stock-movements/{movement_id}
- GET /api/stock-movements?operatorCode=WIK&locationId=X&skuId=Y&since=DATE
- GET /api/stock-reason-codes?operatorCode=WIK&active=true
- GET /api/stock-reservations?woId=X
- PATCH /api/stock-locations/{location_id}
- PATCH /api/stock-reason-codes/{operator}/{code}
- POST /api/stock-availability
- POST /api/stock-cycle-counts
- POST /api/stock-locations
- POST /api/stock-movements
- POST /api/stock-reason-codes
- POST /api/stock-reservations
- POST /api/stock-transfers

4.2 Tables

- stock_balance
- stock_location
- stock_movement
- stock_reason_code
- stock_reservation

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- assign-contractor-with-stock
- sophix-superadmin
- wik-internal-tech-team

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.

- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group O:** Operations Support Resources | **Version:** v1.0 | **Module:** Stock Chain

The system of record for **equipment stock balances + movements** across Wananchi's two-tier stock topology: central warehouse (one per operator) and contractor vans (one logical stock location per contractor). Every router, ONT, splitter, wall socket, and cable that enters or leaves any location creates a movement row; balances are derived projections. Owned at operator scope. References `equipment_sku.sku_id` from PLM-CFG-06 (this module does not define SKUs); references `contractor_id` from EM-02 (this module does not define contractors).

v1.0 scope: **quantity tracking only at the SKU level — no individual equipment instance / serial number tracking.** Serialized instance tracking is deferred to a future DD (OSR-INSTANCE-01 or equivalent). Returns + RMA workflows in v1.0 work at the SKU + qty level.

1. What this module is

When a vendor delivers 200 Huawei HG8245H5 ONTs to Wananchi's central warehouse in Nairobi, the system needs to know: this SKU now has +200 quantity at this location. When 20 of those move out to franchise contractor `contractor_ke_nairobi_a`'s van, the system needs to know: -20 at central, +20 at the van. When the installer at the van installs one at Hon. Sarah Wanjiku's house, the system needs to know: -1 at the van, and the customer's WO is bound to the install event. When she returns the equipment 18 months later at disconnect, +1 to the van. Every one of these events is a

movement; every movement adjusts balances; the movement log is the truth and the balance is a cache.

The two-tier topology — central warehouse + contractor vans — is intentional. Wananchi operates in markets where field operations rely heavily on contractor networks; each contractor needs visible stock that they can dispatch from without round-tripping through central. The operator (Wananchi central ops) sees both tiers; the contractor sees only their van.

What this module does

1. **Maintain stock locations** — operator-scoped, two-tier: one `CENTRAL_WAREHOUSE` per operator, plus `N CONTRACTOR_VAN` locations (one per contractor with active coverage in EM-02).
2. **Record stock movements** — every quantity change is an immutable movement row with reason code, qty, source/target locations, references to the originating business object (vendor delivery #, WO ID, return ID, write-off form #).
3. **Project balances** — `stock_balance(sku_id, location_id, qty_on_hand)` is a derived materialized projection of all movements, refreshed asynchronously after each commit (and on demand via API).
4. **Validate stock availability** — at WO assignment time, callers ask "does contractor X's van have a Huawei HG8245H5 in stock?" Yes/no answer with current qty.
5. **Reserve stock for committed WOs** — when a WO is finalized but not yet installed, the qty is "reserved" — visible in balance but unavailable for other WOs. Resolved on install (movement `RECORDED`) or WO cancellation (reservation released).
6. **Emit movement events** — every balance-changing event publishes to Kafka for downstream consumers (BIL-01 deposit charge, ops dashboards, EM-02 contractor performance metrics).
7. **Support cycle counts + adjustments** — periodic physical inventory reconciliation produces `ADJUSTED` movements with audit trail (who counted, when, why the variance).

What this module does NOT do

- **Doesn't define SKUs.** SKU catalog (brand, model, ownership semantics, fees, kind, compatibility) lives in PLM-CFG-06. OSR-01 references `equipment_sku.sku_id` as FK; never duplicates SKU metadata.
- **Doesn't track individual equipment instances.** Serial number tracking, per-instance lifecycle (received → in-van → installed → returned → RMA → written-off), per-instance warranty, per-instance customer binding — all deferred to a future DD. v1.0 works at the (sku_id, location_id, qty) level only. If a customer returns a defective ONT, OSR-01 records +1 to the contractor van's balance for that SKU; **which specific physical ONT was returned is not knowable in v1.0.**
- **Doesn't define contractors.** Contractor entity, scope per region, skills, availability slots, employed staff — all EM-02. OSR-01 references `contractor_id` from EM-02 to identify which van is which. v1.0 ships a minimal stub `osr_contractor_stub(contractor_id PK, contractor_name)` to satisfy FK references; retired when EM-02 lands.
- **Doesn't manage procurement.** Vendor relationships, purchase orders, vendor SLAs, lead times — out of scope. A separate Procurement DD (pending) owns those; OSR-01 receives "vendor X delivered N units of SKU Y on date Z" as input and records the movement.
- **Doesn't handle RMA / repair workflows.** RMA process orchestration (sending failed equipment back to vendor, tracking repair status, receiving refurbished units back) is a future DD. v1.0 records the inbound "returned from customer" movement and the outbound "written-off" movement; the in-between is out of scope.
- **Doesn't track equipment-on-customer-premises.** Once an installer completes installation, OSR-01's record is "the SKU left the van as `INSTALLED` with reference to WO X". Where the physical equipment sits between install and disconnect is not OSR-01's concern in v1.0; the customer's account in ILM-CFG-01 carries the installed-equipment summary via the WO trail. (When serialized tracking lands, equipment-on-customer-premises will be the `INSTALLED` instance state.)
- **Doesn't gate WO finalization.** WO module decides whether to proceed with a WO based on the OSR-01 stock-availability response, but the gating logic + business rules live in WO module's Drools rules, not here. OSR-01 just answers "qty available" honestly.

Regional flexibility

Stock chain operations are similar across operators (receive, transfer, install, return, write-off) but the operational scale and reporting cadence differ:

- **WIK (Kenya)** — high volume; 24 active contractor vans across Nairobi + Mombasa + Diani; cycle counts quarterly on central, monthly per van.

- **WUG (Uganda)** — smaller scale; ~10 active vans; cycle counts quarterly across the board.
- **WTZ (Tanzania)** — smaller still; cycle count cadence TBD per operator.
- **YASSN (Senegal)** — Dakar-centric; ~6 active vans; cycle counts monthly per van.

The schema accommodates all four with operator-scoped data + per-operator config for cycle-count thresholds. v1.0 ships with sensible defaults; operator-config overrides are out of v1.0 scope (future enhancement).

Glossary

Term	Meaning
Stock location	A logical place where stock sits. Two kinds: <code>CENTRAL_WAREHOUSE</code> (one per operator) and <code>CONTRACTOR_VAN</code> (one per active contractor).
Movement	A single quantity change for one SKU at one location, with a reason code, source/target, qty, references to the originating event. Immutable once committed.
Balance	The derived (<code>sku_id</code> , <code>location_id</code> , <code>qty_on_hand</code>) projection from the movement log. Refreshed after each movement commit.
Reservation	A reduction in "available qty" against a balance, held for a finalized WO that hasn't yet completed installation. Counted as on-hand but not available for new commitments. Released on install or WO cancellation.
Transfer	A movement with <code>direction=TRANSFER</code> and both source and target locations set, recording stock leaving one location and arriving at another in a single atomic row. Multi-SKU transfers (e.g., 20 ONTs + 15 routers shipped together) share a <code>batch_id</code> across rows.
Cycle count	A physical inventory audit at a location, producing one or more <code>ADJUSTED</code> movements when the counted qty differs from the system qty.
Write-off	A movement recording loss, theft, damage, expiry, or obsolescence — qty leaves the system without going to a customer or vendor.
Reason code	An operator-extensible code categorising why a movement happened. Used for reporting and contractor performance metrics.
Source / target	For a transfer, the FROM and TO locations. For a customer install (<code>CONTRACTOR_VAN</code> → customer), source is the van; target is conceptually "the customer" but recorded as a NULL target with a <code>wo_id</code> reference.

2. Tables overview + relationships

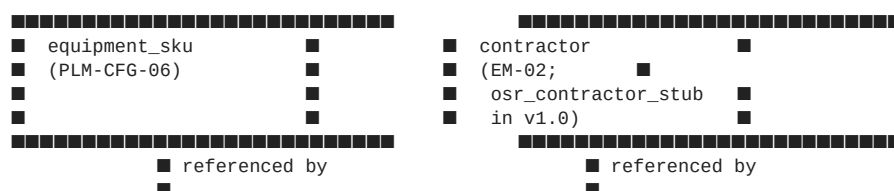
2.1 Tables overview

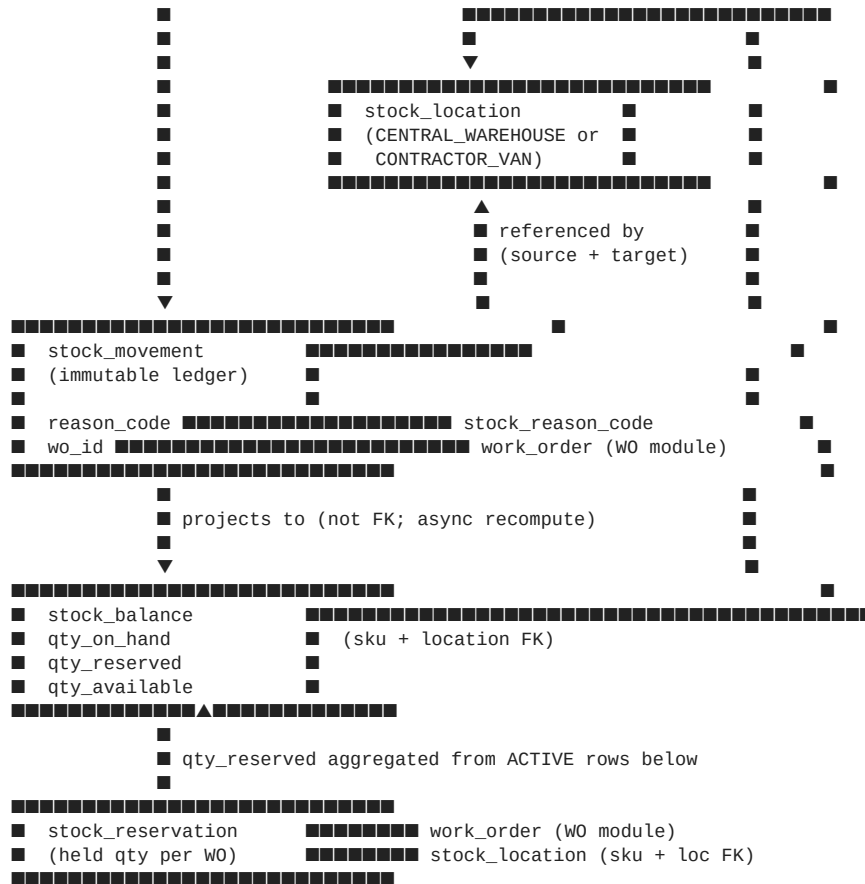
OSR-01 owns five tables:

Table	Purpose
<code>stock_location</code>	Operator-scoped catalog of stock locations (central warehouses + contractor vans).
<code>stock_movement</code>	The immutable ledger — every quantity change as one row. Source of truth for all balances.
<code>stock_balance</code>	Derived projection: current <code>qty_on_hand</code> per (<code>sku_id</code> , <code>location_id</code>). Asynchronously refreshed from movements.
<code>stock_reservation</code>	Holds committed-but-not-yet-installed qty per WO, deducted from "available qty".
<code>stock_reason_code</code>	Operator-extensible catalog of reason codes used on movements (vendor receipt, install, return, write-off, etc.).

Full DDL in §3.

2.2 Entity relationships





Reading the diagram:

- **Solid ■■■■■■ arrows** are foreign-key references, pointing from the table that holds the FK to the table being referenced.
- **projects** to between `stock_movement` and `stock_balance` is NOT a foreign key. `stock_balance` is a denormalised projection: `qty_on_hand` is recomputed asynchronously after each commit by summing the relevant `stock_movement` rows. There is no FK from balance back to a specific movement row — balance is the running total.
- `stock_reason_code` is referenced by `stock_movement.reason_code` (FK). Both tables carry a `movement_direction` column for the same reason — it's defined on the reason code and **denormalised onto the movement row** for query convenience (so filters like "all OUTBOUND movements in May" don't need to join). Updates to a reason code's direction would require backfill of historical movements; v1.0 treats reason codes as effectively immutable once used.
- `stock_reservation.qty` values flow into `stock_balance.qty_reserved` as an aggregate (sum of ACTIVE reservations for the same sku+location), not as a per-row FK.
- The **two FKs into `stock_location` from `stock_movement`** (`source_location_id` and `target_location_id`) are shown collapsed as one arrow; in DDL they are two separate columns.
- **work_order** (in WO module) is referenced by both `stock_movement.wo_id` (when reason is INSTALL or RETURN) and `stock_reservation.wo_id`; OSR-01 does not own this entity.

The movement table is the only place quantity actually changes; balance + reservation are derived. A movement commit triggers (async) recompute of the affected (sku_id, location_id) balance. The system is eventually-consistent for balance reads; strictly consistent for movement reads.

2.3 Cross-DD anchoring

- **PLM-CFG-06** — every OSR-01 row referencing a SKU FKs to `equipment_sku.sku_id`. PLM-CFG-06 is upstream.
- **EM-02** — `contractor_id` referenced on `stock_location` for vans; v1.0 uses stub. WO module also reads EM-02 for contractor identity.
- **WO module** — WO finalization triggers a reservation; WO completion triggers the install movement that consumes the reservation. WO cancellation releases the reservation.

- **BIL-01 charging** — install movements emit Kafka events that BIL-01 consumes to apply equipment deposit + monthly rental charges (per PLM-CFG-06 R-PLM-EQ-4).
- **ILM-CFG-01** — return movements at disconnect feed into the customer-account flag system (per ILM-CFG-01 §3.5) for "equipment-not-yet-returned" flags.
- **Procurement DD (pending)** — vendor receipt movements consume PO references from procurement; v1.0 carries a free-text vendor_reference field.

3. Schemas

3.1 stock_location

```
CREATE TABLE stock_location (
  location_id          TEXT PRIMARY KEY,          -- 'WIK-CENTRAL-NRB' | 'WIK-VAN-contractor_ke_nairobi_a' – human
  operator_code        TEXT NOT NULL,
  location_kind        TEXT NOT NULL,            -- 'CENTRAL_WAREHOUSE' | 'CONTRACTOR_VAN'
  contractor_id        TEXT NULL,               -- FK to EM-02 contractor (or osr_contractor_stub); REQUIRED iff
  display_name_primary TEXT NOT NULL,           -- 'WIK Central Warehouse – Nairobi' | 'Franchise A Van (Lavington /
  display_name_secondary TEXT NULL,             -- bilingual where applicable
  address              TEXT NULL,              -- physical address; meaningful for central, less so for vans
  active              BOOLEAN NOT NULL DEFAULT TRUE,
  notes               TEXT NULL,
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_by          TEXT NOT NULL,
  CHECK ((location_kind = 'CENTRAL_WAREHOUSE' AND contractor_id IS NULL) OR
        (location_kind = 'CONTRACTOR_VAN' AND contractor_id IS NOT NULL))
);

CREATE INDEX idx_location_operator ON stock_location (operator_code, active);
CREATE INDEX idx_location_kind ON stock_location (operator_code, location_kind, active);
CREATE INDEX idx_location_contractor ON stock_location (contractor_id) WHERE contractor_id IS NOT NULL;
CREATE UNIQUE INDEX uq_location_central
  ON stock_location (operator_code, location_kind) WHERE location_kind = 'CENTRAL_WAREHOUSE';
CREATE UNIQUE INDEX uq_location_van
  ON stock_location (contractor_id) WHERE location_kind = 'CONTRACTOR_VAN' AND active = TRUE;
```

The partial UNIQUE indexes enforce: exactly one active CENTRAL_WAREHOUSE per operator, and at most one active CONTRACTOR_VAN per contractor.

Sample data:

location_id	operator_code	location_kind	contractor_id	display_name_primary	active
WIK-CENTRAL-NRB	WIK	CENTRAL_WAREHOUSE	NULL	WIK Central Warehouse – Nairobi	true
WIK-VAN-contractor_ke_nairobi_a	WIK	CONTRACTOR_VAN	contractor_ke_nairobi_a	Franchise A Van (Lavington / Westlands)	true
WIK-VAN-contractor_ke_nairobi_b	WIK	CONTRACTOR_VAN	contractor_ke_nairobi_b	Franchise B Van (Lavington backup)	true
WIK-VAN-contractor_ke_nairobi_e	WIK	CONTRACTOR_VAN	contractor_ke_nairobi_e	Franchise E Van (Eastlands)	true
WIK-VAN-contractor_ke_nairobi_k	WIK	CONTRACTOR_VAN	contractor_ke_nairobi_k	Franchise K Van (Kibera / Mukuru)	true
WIK-VAN-wik-internal-tech-team	WIK	CONTRACTOR_VAN	wik-internal-tech-team	WIK Internal Tech Team Stock (Karen)	true
WIK-VAN-contractor_ke_mombasa_2	WIK	CONTRACTOR_VAN	contractor_ke_mombasa_2	Mombasa Franchise Van (Nyali + Diani)	true
WIK-VAN-contractor_ke_mombasa_1	WIK	CONTRACTOR_VAN	contractor_ke_mombasa_1	Former Mombasa Franchise (retired)	false
YASSN-CENTRAL-DKR	YASSN	CENTRAL_WAREHOUSE	NULL	YASSN Central Warehouse – Dakar	true
YASSN-VAN-contractor_sn_dakar_1	YASSN	CONTRACTOR_VAN	contractor_sn_dakar_1	Franchise principale Dakar centre	true

location_id	operator_code	location_kind	contractor_id	display_name_primary	active
WUG-CENTRAL-KLA	WUG	CENTRAL_WAREHOUSE	NULL	WUG Central Warehouse – Kampala	true
WTZ-CENTRAL-DSM	WTZ	CENTRAL_WAREHOUSE	NULL	WTZ Central Warehouse – Dar es Salaam	true

Row 8 — contractor_ke_mombasa_1 van is active=false matching the contractor coverage termination in ILM-CFG-02 sample data (contractor relationship ended 2024-12-31). The van's historical balance is preserved; new movements are blocked.

Row 6 — wik-internal-tech-team has its own van (internal pseudo-contractor pattern from ILM-CFG-02): internal staff also stock from central and dispatch to customers via their own logical "van" location.

3.2 stock_reason_code

```
CREATE TABLE stock_reason_code (
  operator_code      TEXT NOT NULL,
  reason_code        TEXT NOT NULL,
  movement_direction TEXT NOT NULL,
  display_name_primary TEXT NOT NULL,
  display_name_secondary TEXT NULL,
  affects_available_qty BOOLEAN NOT NULL DEFAULT TRUE,
  requires_approval  BOOLEAN NOT NULL DEFAULT FALSE,
  active             BOOLEAN NOT NULL DEFAULT TRUE,
  notes              TEXT NULL,
  created_at         TIMESTAMP NOT NULL DEFAULT NOW(),
  PRIMARY KEY (operator_code, reason_code)
);

CREATE INDEX idx_reason_active ON stock_reason_code (operator_code, active);
```

Sample data:

operator_code	reason_code	movement_direction	display_name_primary	requires_approval	active
WIK	VENDOR_RECEIPT	INBOUND	Vendor delivery received	false	true
WIK	TRANSFER_OUT	TRANSFER	Transferred to contractor van	false	true
WIK	TRANSFER_IN	TRANSFER	Transferred from contractor van back to central	false	true
WIK	INSTALL	OUTBOUND	Installed at customer premises	false	true
WIK	RETURN	INBOUND	Returned by customer at disconnect	false	true
WIK	WRITE_OFF_DAMAGE	OUTBOUND	Written off – damaged in transit / install	true	true
WIK	WRITE_OFF_LOSS	OUTBOUND	Written off – lost or stolen	true	true
WIK	WRITE_OFF_OBSOLETE	OUTBOUND	Written off – model deprecated	true	true
WIK	CYCLE_COUNT_ADJUSTMENT_POS	INBOUND	Cycle count – positive variance	true	true
WIK	CYCLE_COUNT_ADJUSTMENT_NEG	OUTBOUND	Cycle count – negative variance	true	true
WIK	RECLAIM_FROM_VAN	TRANSFER	Operator-initiated pullback from van	true	true
YASSN	VENDOR_RECEIPT	INBOUND	Livraison fournisseur reçue	false	true
YASSN	INSTALL	OUTBOUND	Installé chez le client	false	true
YASSN	RETURN	INBOUND	Retourné par le client	false	true

Reason codes are operator-extensible; new ones land via INSERT, no DDL. `requires_approval = true` reason codes need a second-person approver recorded in the movement row (`approved_by`); v1.0 enforces this in API but not at DB level.

3.3 stock_movement

The immutable ledger. Every balance change is exactly one row here.

```
CREATE TABLE stock_movement (
  movement_id      TEXT PRIMARY KEY,                -- 'mov_01HZ...' ULID
  operator_code     TEXT NOT NULL,
  sku_id           TEXT NOT NULL REFERENCES equipment_sku(sku_id),
  reason_code       TEXT NOT NULL,                  -- FK composite (operator_code, reason_code) → stock_reason_code
  movement_direction TEXT NOT NULL,                -- denormalized from reason_code for query convenience
  qty              INTEGER NOT NULL,               -- always positive; sign is implied by movement_direction
  source_location_id TEXT NULL REFERENCES stock_location(location_id), -- where stock left from
  target_location_id TEXT NULL REFERENCES stock_location(location_id), -- where stock arrived
  batch_id         TEXT NULL,                      -- groups multi-SKU operations (one transfer, one cycle count, or
  wo_id           TEXT NULL,                       -- WO module reference (for INSTALL or RETURN movements)
  vendor_reference TEXT NULL,                      -- vendor PO / delivery note ref (for VENDOR_RECEIPT)
  approved_by      TEXT NULL,                     -- user_id of approver (for requires_approval reason codes)
  notes           TEXT NULL,
  created_at       TIMESTAMP NOT NULL DEFAULT NOW(),
  created_by      TEXT NOT NULL,                  -- user_id of person/service who created the movement
  -- immutability enforced at app layer + via trigger preventing UPDATE
  FOREIGN KEY (operator_code, reason_code) REFERENCES stock_reason_code(operator_code, reason_code),
  CHECK (qty > 0),
  CHECK ((movement_direction = 'INBOUND' AND target_location_id IS NOT NULL AND source_location_id IS NULL) OR
        (movement_direction = 'OUTBOUND' AND source_location_id IS NOT NULL AND target_location_id IS NULL) OR
        (movement_direction = 'TRANSFER' AND source_location_id IS NOT NULL AND target_location_id IS NOT NULL))
);

CREATE INDEX idx_movement_sku_target ON stock_movement (sku_id, target_location_id, created_at);
CREATE INDEX idx_movement_sku_source ON stock_movement (sku_id, source_location_id, created_at);
CREATE INDEX idx_movement_operator ON stock_movement (operator_code, created_at DESC);
CREATE INDEX idx_movement_reason ON stock_movement (operator_code, reason_code, created_at DESC);
CREATE INDEX idx_movement_wo ON stock_movement (wo_id) WHERE wo_id IS NOT NULL;
CREATE INDEX idx_movement_batch ON stock_movement (batch_id) WHERE batch_id IS NOT NULL;
```

A **transfer** is a single row with `movement_direction='TRANSFER'`, both `source_location_id` and `target_location_id` set, and `qty=N`. The `qty` leaves source and arrives at target atomically. **Multi-SKU operations** (a single transfer covering 20 ONTs + 15 routers; a single cycle count adjusting multiple SKUs; one vendor receipt mixing multiple SKUs on the same PO) are modeled by giving each individual SKU+qty row the same `batch_id`. Single-SKU movements leave `batch_id` NULL.

INSTALL and RETURN rows are single-row movements with only one location set (the van) — the customer is not a tracked location in v1.0.

Sample data:

movement_id	sku_id	reason_code	movement_direction	qty	source_location_id	target_location_id	batch_id	wo_id	vendor_reference	created_at
mov_01HZ_VENDOR_HG8245_200	WIK-ONT-HG8245-H5	VENDOR_RECEIPT	INBOUND	200	NULL	WIK-CENTRAL-NRB	batch_01HZ_P0_0123	NULL	P0-WIK-2025-Q1-0123	2025-01-15 10:00:00
mov_01HZ_VENDOR_HAP_AC2_150	WIK-RTR-HAP-AC2	VENDOR_RECEIPT	INBOUND	150	NULL	WIK-CENTRAL-NRB	batch_01HZ_P0_0123	NULL	P0-WIK-2025-Q1-0123	2025-01-15 10:00:00
mov_01HZ_XFER_HG8245_20	WIK-ONT-HG8245-H5	TRANSFER_OUT	TRANSFER	20	WIK-CENTRAL-NRB	WIK-VAN-contractor_ke_nairobi_a	batch_01HZ_Z_NRB_A_001	NULL	NULL	2025-02-01 08:30:00
mov_01HZ_XFER_HAP_15	WIK-RTR-HAP-AC2	TRANSFER_OUT	TRANSFER	15	WIK-CENTRAL-NRB	WIK-VAN-contractor_ke_nairobi_a	batch_01HZ_Z_NRB_A_001	NULL	NULL	2025-02-01 08:30:00
mov_01HZ_INSTALL_HG8245_WANJIKU	WIK-ONT-HG8245-H5	INSTALL	OUTBOUND	1	WIK-VAN-wik-internal-tech-team	NULL	NULL	wo_01HZ_WANJIKU_INSTALL	NULL	2026-02-23 15:45:00
mov_01HZ_INSTALL_HAP_WANJIKU	WIK-RTR-HAP-AC2	INSTALL	OUTBOUND	1	WIK-VAN-wik-internal-tech-team	NULL	NULL	wo_01HZ_WANJIKU_INSTALL	NULL	2026-02-23 15:45:00

movement_id	sku_id	reason_code	movement_direction	qty	source_location_id	target_location_id	batch_id	wo_id	vendor_reference	created_at
mov_01HZ_RETURN_HG8245_KIMANI	WIK-ONT-HG8245-H5	RETURN	INBOUND	1	NULL	WIK-VAN-contractor_ke_nairobi_k	NULL	wo_01HZ_KIMANI_DISCONNECT	NULL	2026-05-10 11:20:00
mov_01HZ_WRITE_OFF_HG8245_DAMAGE	WIK-ONT-HG8245-H5	WRITE_OFF_DAMAGE	OUTBOUND	2	WIK-VAN-contractor_ke_nairobi_a	NULL	NULL	NULL	NULL	2026-04-15 14:00:00
mov_01HZ_CYCLE_NEG_HAP_A	WIK-RTR-HAP-AC2	CYCLE_COUNT_ADJUSTMENT_NEG	OUTBOUND	1	WIK-VAN-contractor_ke_nairobi_a	NULL	batch_01HZ_CC_NRB_A_2026_05	NULL	NULL	2026-05-01 09:00:00

Rows 1 + 2: vendor receipt of 200 ONTs + 150 routers from one PO; both rows share batch_id = batch_01HZ_PO_0123. The two rows are atomically committed.

Rows 3 + 4: monthly replenishment transfer to Franchise A van — 20 ONTs + 15 routers. Both rows have direction TRANSFER, source = central, target = van, and share batch_id = batch_01HZ_NRB_A_001.

Rows 5 + 6: install at Hon. Wanjiku's premises — one ONT + one router consumed from the internal team van. wo_id references the install WO. target_location_id is NULL because the customer is not a tracked location in v1.0. batch_id is NULL since these are independent line items on the same WO (either choice is valid).

Row 7: Peter Kimani disconnect — ONT returned, +1 to Franchise K van. Customer-to-van flow is direction INBOUND with NULL source.

Row 8: 2 ONTs written off as damaged at the Franchise A van location. requires_approval=true for write-off reason codes; approved_by would be populated in production (omitted in sample for brevity).

Row 9: cycle count at Franchise A van found 1 fewer router than the system showed; negative variance recorded. batch_id groups all variances from the same cycle count session.

3.4 stock_balance

Derived projection. Refreshed asynchronously after each movement commit.

```
CREATE TABLE stock_balance (
  sku_id          TEXT NOT NULL REFERENCES equipment_sku(sku_id),
  location_id     TEXT NOT NULL REFERENCES stock_location(location_id),
  operator_code   TEXT NOT NULL,
  qty_on_hand     INTEGER NOT NULL DEFAULT 0,
  qty_reserved    INTEGER NOT NULL DEFAULT 0,
  qty_available   INTEGER GENERATED ALWAYS AS (qty_on_hand - qty_reserved) STORED,
  last_movement_at  TIMESTAMP NULL,
  last_recomputed_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  PRIMARY KEY (sku_id, location_id),
  CHECK (qty_on_hand >= 0)
);

CREATE INDEX idx_balance_location ON stock_balance (location_id, qty_available);
CREATE INDEX idx_balance_operator ON stock_balance (operator_code, sku_id);
CREATE INDEX idx_balance_low_stock ON stock_balance (location_id) WHERE qty_available < 5;
```

The qty_available generated column is the right field to consult when asking "can we use one of these?" — it accounts for already-committed-but-not-yet-installed WOs.

Sample data (after the movements above):

sku_id	location_id	qty_on_hand	qty_reserved	qty_available	last_movement_at
WIK-ONT-HG8245-H5	WIK-CENTRAL-NRB	180	0	180	2025-02-01 08:30:00
WIK-ONT-HG8245-H5	WIK-VAN-contractor_ke_nairobi_a	17	3	14	2026-05-01 09:00:00
WIK-ONT-HG8245-H5	WIK-VAN-wik-internal-tech-team	4	1	3	2026-02-23 15:45:00
WIK-ONT-HG8245-H5	WIK-VAN-contractor_ke_nairobi_k	8	0	8	2026-05-10 11:20:00
WIK-RTR-HAP-AC2	WIK-CENTRAL-NRB	135	0	135	2025-02-01 08:30:00

sku_id	location_id	qty_on_hand	qty_reserved	qty_available	last_movement_at
WIK-RTR-HAP-AC2	WIK-VAN-contractor_ke_nairobi_a	13	2	11	2026-05-01 09:00:00
WIK-RTR-HAP-AC2	WIK-VAN-wik-internal-tech-team	6	0	6	2026-02-23 15:45:00

Row 2: Franchise A van has 17 ONTs on hand, 3 reserved (3 finalized WOs not yet installed), so 14 effectively available for new WO assignments. Row 3: internal tech team van has 4 ONTs after the install for Hon. Wanjiku (5 before, -1 after); 1 reserved for an upcoming WO.

The qty_reserved column is updated by the reservation lifecycle (§3.5); qty_on_hand is updated by the balance-recompute job triggered after each movement commit.

3.5 stock_reservation

Holds committed qty per WO from WO finalization to install completion (or WO cancellation).

```
CREATE TABLE stock_reservation (
  reservation_id      TEXT PRIMARY KEY,          -- 'rsv_01HZ...' ULID
  operator_code       TEXT NOT NULL,
  sku_id              TEXT NOT NULL REFERENCES equipment_sku(sku_id),
  location_id         TEXT NOT NULL REFERENCES stock_location(location_id),
  qty                 INTEGER NOT NULL,
  wo_id               TEXT NOT NULL,             -- the WO this reservation is held for
  status              TEXT NOT NULL,             -- 'ACTIVE' | 'CONSUMED' | 'RELEASED' | 'EXPIRED'
  expires_at          TIMESTAMP NULL,            -- after which reservation auto-released
  consumed_at         TIMESTAMP NULL,            -- when WO install completed and INSTALL movement was created
  released_at         TIMESTAMP NULL,            -- when WO was cancelled and reservation was released
  consumed_movement_id TEXT NULL,                -- the INSTALL movement that consumed this reservation
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  created_by          TEXT NOT NULL,
  CHECK (qty > 0),
  CHECK ((status = 'CONSUMED' AND consumed_at IS NOT NULL AND consumed_movement_id IS NOT NULL) OR
        (status = 'RELEASED' AND released_at IS NOT NULL) OR
        (status = 'EXPIRED' AND expires_at IS NOT NULL AND consumed_at IS NULL AND released_at IS NULL) OR
        (status = 'ACTIVE' AND consumed_at IS NULL AND released_at IS NULL))
);

CREATE INDEX idx_reservation_wo      ON stock_reservation (wo_id);
CREATE INDEX idx_reservation_active  ON stock_reservation (sku_id, location_id) WHERE status = 'ACTIVE';
CREATE INDEX idx_reservation_expiring ON stock_reservation (expires_at) WHERE status = 'ACTIVE' AND expires_at IS NOT NULL;
```

Lifecycle:

1. WO module finalizes WO → calls POST /api/stock-reservations for each equipment line → reservation row inserted with status=ACTIVE. stock_balance.qty_reserved += qty.
2. Install completes → WO module calls POST /api/stock-movements with reason INSTALL referencing the WO → OSR-01 transitions matching reservations to status=CONSUMED, consumed_movement_id=<new movement>. stock_balance.qty_reserved -= qty (atomic with the movement's qty_on_hand -= qty, so qty_available net change is zero — the install was already accounted for via reservation).
3. WO cancelled before install → WO module calls DELETE /api/stock-reservations/{id} → reservation transitions to status=RELEASED. stock_balance.qty_reserved -= qty.
4. Expiry — a reservation with expires_at IS NOT NULL AND expires_at < NOW() and still ACTIVE is auto-transitioned by a periodic job (reservation-expiry-sweeper-<operator>-v<n>, every 10 min) to EXPIRED. The WO is flagged for review (it has no committed stock anymore).

Sample data:

reservation_id	sku_id	location_id	qty	wo_id	status	created_at	consumed_at
rsv_01HZ_WANJIKU_HG8245	WIK-ONT-HG8245-H5	WIK-VAN-wik-internal-tech-team	1	wo_01HZ_WANJIKU_INSTALL	CONSUMED	2026-02-17 10:00:00	2026-02-23 15:45:00
rsv_01HZ_WANJIKU_HAP	WIK-RTR-HAP-AC2	WIK-VAN-wik-internal-tech-team	1	wo_01HZ_WANJIKU_INSTALL	CONSUMED	2026-02-17 10:00:00	2026-02-23 15:45:00
rsv_01HZ_FRANCHISE_A_3_PENDING	WIK-ONT-HG8245-H5	WIK-VAN-contractor_ke_nairobi_a	3	wo_01HZ_FRANCHISE_A_BATCH_001	ACTIVE	2026-05-18 14:00:00	NULL

reservation_id	sku_id	location_id	qty	wo_id	status	created_at	consumed_at
rsv_01HZ_FRANCHISE_A_2_RTR	WIK-RTR-HAP-A C2	WIK-VAN-contractor_ke_nairobi_a	2	wo_01HZ_FRANCHISE_A_BATCH_002	ACTIVE	2026-05-19 09:00:00	NULL

Rows 1 + 2: the reservations made for Hon. Wanjiku's install. Both consumed on 2026-02-23 when the INSTALL movements were created.

Rows 3 + 4: Franchise A has 3 ONTs and 2 routers reserved for in-flight WOs — those are the qty_reserved values shown in the §3.4 balance table.

4. Rules

Rule	Description
R-OSR-SC-1	The movement log is immutable: stock_movement rows are append-only. UPDATE on a movement row is blocked by trigger. To correct a wrong movement, insert a compensating movement (e.g., a CYCLE_COUNT_ADJUSTMENT_POS to add back qty wrongly written off). The original row stays.
R-OSR-SC-2	Every balance change derives from movements: stock_balance.qty_on_hand for (sku_id, location_id) = sum of all INBOUND movements with target_location_id = location_id and INBOUND-direction TRANSFER movements with target_location_id = location_id, minus all OUTBOUND movements with source_location_id = location_id and OUTBOUND-direction TRANSFER movements with source_location_id = location_id. Recomputation is eventually-consistent, triggered after each movement commit.
R-OSR-SC-3	TRANSFER movements have both source AND target location set. INBOUND movements (vendor receipt, return from customer, cycle-count positive variance) have only target set. OUTBOUND movements (install, write-off, cycle-count negative variance) have only source set. The CHECK constraint enforces this.
R-OSR-SC-4	Two-tier topology enforcement: TRANSFER movements MUST go from CENTRAL_WAREHOUSE to CONTRACTOR_VAN or from CONTRACTOR_VAN to CENTRAL_WAREHOUSE. Van-to-van transfers are NOT supported in v1.0 (would route through central). Validated at API; future enhancement may relax.
R-OSR-SC-5	A reservation deducts from qty_available (via qty_reserved) but not from qty_on_hand. The reserved stock is still physically at the location; it's just committed to a specific WO. Multiple reservations can stack on the same location until they hit qty_on_hand.
R-OSR-SC-6	Reservation consumption is atomic with the consuming movement: in the same transaction that creates the INSTALL movement, matching reservations are transitioned to CONSUMED. A reservation can be matched only if it (a) has the same wo_id as the movement, (b) is currently ACTIVE, (c) covers the same (sku_id, location_id) as the movement's source. Quantity matching is exact — a 1-unit movement can consume exactly one 1-unit reservation.
R-OSR-SC-7	Reservation expiry: a reservation with expires_at < NOW() and still ACTIVE is auto-expired by the periodic sweeper. Default expires_at = created_at + 30 days if not specified by caller. Expired reservations release their qty_reserved contribution. The associated WO is flagged for review (no committed stock; can't install).
R-OSR-SC-8	Negative qty is forbidden: stock_balance.qty_on_hand is constrained ≥ 0. Movements that would drive a balance negative are rejected by the API with INSUFFICIENT_STOCK (e.g., trying to write off 5 ONTs from a van that only has 3).
R-OSR-SC-9	Write-off and cycle-count adjustment movements require approved_by per the reason code's requires_approval = true flag. API rejects movements without an approver for those reason codes. v1.0 does not enforce approver authority hierarchy (any authenticated user can approve); future enhancement may require role check.
R-OSR-SC-10	Stock query semantics: GET /api/stock-balances?location=X&sku=Y returns the latest committed balance. Stock availability queries (GET /api/stock-availability/{wo_context}) read qty_available which reflects active reservations. Both are eventually-consistent; lag is typically <1s but can be longer under high movement volume.
R-OSR-SC-11	Operator-scoped queries by default: all REST endpoints scope to the requesting actor's operator unless they hold sophix-superadmin role. Cross-operator stock visibility is not granted to operator-local roles.

Rule	Description
R-OSR-SC-12	Balance recompute integrity: after each movement commit, a background worker recomputes <code>stock_balance.qty_on_hand</code> for the affected (<code>sku_id</code> , <code>location_id</code>) rows. If multiple movements commit in quick succession, the recompute coalesces (only the latest state is written). A nightly job verifies balance integrity by recomputing all balances from scratch and comparing to stored values; discrepancies trigger ops alerts.

5. REST API

5.1 Stock balance + availability

The two most-called endpoints. Stock-balance returns raw on-hand + reserved + available; stock-availability is a quick boolean check used by WO routing.

```
GET /api/stock-balances?operatorCode=WIK&locationId=WIK-VAN-contractor_ke_nairobi_a HTTP/1.1
Authorization: Bearer <staff-or-service-jwt>
```

```
→ 200 OK
{
  "locationId":      "WIK-VAN-contractor_ke_nairobi_a",
  "operatorCode":    "WIK",
  "asOf":            "2026-05-20T14:30:00Z",
  "balances": [
    {
      "skuId":        "WIK-ONT-HG8245-H5",
      "qtyOnHand":    17,
      "qtyReserved":  3,
      "qtyAvailable": 14,
      "lastMovementAt": "2026-05-01T09:00:00Z"
    },
    {
      "skuId":        "WIK-RTR-HAP-AC2",
      "qtyOnHand":    13,
      "qtyReserved":  2,
      "qtyAvailable": 11,
      "lastMovementAt": "2026-05-01T09:00:00Z"
    }
  ],
  "totalSkus": 2
}
```

```
POST /api/stock-availability HTTP/1.1
Authorization: Bearer <wo-service-jwt>
Content-Type: application/json
```

```
{
  "operatorCode":  "WIK",
  "locationId":    "WIK-VAN-wik-internal-tech-team",
  "requirements": [
    { "skuId": "WIK-ONT-HG8245-H5", "qty": 1 },
    { "skuId": "WIK-RTR-HAP-AC2",   "qty": 1 }
  ]
}

→ 200 OK
{
  "locationId":  "WIK-VAN-wik-internal-tech-team",
  "available":   true,
  "details": [
    { "skuId": "WIK-ONT-HG8245-H5", "requested": 1, "available": 3, "ok": true },
    { "skuId": "WIK-RTR-HAP-AC2",   "requested": 1, "available": 6, "ok": true }
  ]
}
```

Or when insufficient:

```
{
  "locationId":  "WIK-VAN-contractor_ke_nairobi_k",
  "available":   false,
  "details": [
    { "skuId": "WIK-ONT-HG8245-H5", "requested": 5, "available": 8, "ok": true },
    { "skuId": "WIK-RTR-HAP-AC2",   "requested": 3, "available": 0, "ok": false }
  ]
}
```

The WO module's Drools assign-contractor-with-stock rule reads this response and may pick a backup contractor whose van has stock.

5.2 Movements

```
POST /api/stock-movements HTTP/1.1
Authorization: Bearer <service-jwt>
Content-Type: application/json

{
  "operatorCode":      "WIK",
  "skuId":             "WIK-ONT-HG8245-H5",
  "reasonCode":        "INSTALL",
  "qty":               1,
  "sourceLocationId":  "WIK-VAN-wik-internal-tech-team",
  "targetLocationId":  null,
  "woId":              "wo_01HZ_WANJIKU_INSTALL",
  "notes":             "Install completed by tech_wik_k02"
}

→ 201 Created
{
  "movementId":        "mov_01HZ_INSTALL_HG8245_WANJIKU",
  "balanceAfter": {
    "skuId":           "WIK-ONT-HG8245-H5",
    "locationId":      "WIK-VAN-wik-internal-tech-team",
    "qtyOnHand":       4,
    "qtyReserved":     0,
    "qtyAvailable":    4
  },
  "consumedReservations": ["rsv_01HZ_WANJIKU_HG8245"]
}
```

Note consumedReservations: if matching reservations existed for the same (wo_id, sku_id, location_id), they're transitioned to CONSUMED in the same transaction.

Transfer endpoint:

```
POST /api/stock-transfers HTTP/1.1
Authorization: Bearer <bo-admin-or-service-jwt>
Content-Type: application/json

{
  "operatorCode":      "WIK",
  "sourceLocationId":  "WIK-CENTRAL-NRB",
  "targetLocationId":  "WIK-VAN-contractor_ke_nairobi_a",
  "lines": [
    { "skuId": "WIK-ONT-HG8245-H5", "qty": 20 },
    { "skuId": "WIK-RTR-HAP-AC2",   "qty": 15 }
  ],
  "notes":             "Monthly replenishment"
}

→ 201 Created
{
  "batchId":           "batch_01HZ_NRB_A_002",
  "movements": [
    { "movementId": "mov_01HZ_XFER_HG8245_20", "skuId": "WIK-ONT-HG8245-H5", "qty": 20 },
    { "movementId": "mov_01HZ_XFER_HAP_15",   "skuId": "WIK-RTR-HAP-AC2",   "qty": 15 }
  ]
}
```

A transfer is one row per SKU line, all sharing a batch_id; multi-line transfers are common (vendor delivers a mixed pallet; van replenishment ships multiple SKUs at once).

```
GET /api/stock-movements?operatorCode=WIK&locationId=X&skuId=Y&since=DATE # query the audit log
GET /api/stock-movements/{movement_id} # detail
```

5.3 Reservations

```
POST /api/stock-reservations HTTP/1.1
Authorization: Bearer <wo-service-jwt>
Content-Type: application/json

{
  "operatorCode":      "WIK",
  "skuId":             "WIK-ONT-HG8245-H5",
  "locationId":        "WIK-VAN-wik-internal-tech-team",
  "qty":               1,
  "woId":              "wo_01HZ_WANJIKU_INSTALL",
  "expiresAt":         "2026-03-23T00:00:00Z"
}
```

```

}

→ 201 Created
{
  "reservationId": "rsv_01HZ_WANJIKU_HG8245",
  "status": "ACTIVE",
  "balanceAfter": {
    "skuId": "WIK-ONT-HG8245-H5",
    "locationId": "WIK-VAN-wik-internal-tech-team",
    "qtyOnHand": 5,
    "qtyReserved": 1,
    "qtyAvailable": 4
  }
}

```

Or rejected:

```

→ 409 Conflict
{
  "error": "INSUFFICIENT_AVAILABLE_STOCK",
  "skuId": "WIK-ONT-HG8245-H5",
  "locationId": "WIK-VAN-contractor_ke_nairobi_k",
  "qtyRequested": 5,
  "qtyAvailable": 2,
  "details": "Cannot reserve 5; only 2 available (8 on-hand, 6 already reserved)"
}

DELETE /api/stock-reservations/{reservation_id} # release (used for WO cancellation)
GET /api/stock-reservations?woId=X # query reservations by WO

```

5.4 Location catalog

```

GET /api/stock-locations?operatorCode=WIK&active=true
GET /api/stock-locations/{location_id}
POST /api/stock-locations # create (BO admin); used at contractor onboarding
PATCH /api/stock-locations/{location_id} # update mutable fields; or deactivate

```

5.5 Reason code catalog

```

GET /api/stock-reason-codes?operatorCode=WIK&active=true
POST /api/stock-reason-codes # add operator-extensible reason
PATCH /api/stock-reason-codes/{operator}/{code} # update / deactivate

```

5.6 Cycle count + adjustments

```

POST /api/stock-cycle-counts HTTP/1.1
Authorization: Bearer <bo-admin-jwt>
Content-Type: application/json

{
  "operatorCode": "WIK",
  "locationId": "WIK-VAN-contractor_ke_nairobi_a",
  "countedBy": "ops_user_k01",
  "countedAt": "2026-05-01T09:00:00Z",
  "counts": [
    { "skuId": "WIK-ONT-HG8245-H5", "countedQty": 17, "systemQty": 17 },
    { "skuId": "WIK-RTR-HAP-AC2", "countedQty": 13, "systemQty": 14 }
  ],
  "approvedBy": "ops_supervisor_03",
  "notes": "Monthly cycle count"
}

→ 201 Created
{
  "cycleCountId": "cc_01HZ_NRB_A_2026_05",
  "adjustmentsCreated": [
    {
      "movementId": "mov_01HZ_CYCLE_NEG_HAP_A",
      "skuId": "WIK-RTR-HAP-AC2",
      "qty": 1,
      "direction": "OUTBOUND",
      "reasonCode": "CYCLE_COUNT_ADJUSTMENT_NEG"
    }
  ],
  "noVarianceSkus": ["WIK-ONT-HG8245-H5"]
}

```

The cycle count produces ADJUSTED movements for each variance. Zero-variance SKUs don't get a movement row (the count was just a confirmation).

6. Kafka events

OSR-01 publishes on `sophix.osr.sc.*`.

Event type	Topic	Emitted when	Consumers
StockMovementRecorded	sophix.osr.sc.movement-recorded	Any new stock_movement row	BIL-01 (equipment charges), EM-02 performance metrics, ops dashboards
StockReservationCreated	sophix.osr.sc.reservation-created	New reservation	WO module audit trail
StockReservationConsumed	sophix.osr.sc.reservation-consumed	Reservation → CONSUMED	WO module (install confirmation)
StockReservationReleased	sophix.osr.sc.reservation-released	Reservation → RELEASED (WO cancelled)	WO module
StockReservationExpired	sophix.osr.sc.reservation-expired	Sweeper auto-expired reservation	WO module (flag WO for review), ops alerts
StockLowAlert	sophix.osr.sc.low-alert	A van's qty_available for an is_install_required SKU drops below threshold	Ops dashboards, EM-02 contractor notifications
StockBalanceRecomputed	sophix.osr.sc.balance-recomputed	Balance recompute completed	Internal (ops monitoring)
StockCycleCountCompleted	sophix.osr.sc.cycle-count-completed	Cycle count submitted	Ops dashboards; flagged for review if variance > N%

6.1 Sample event — StockMovementRecorded (install)

```
{
  "eventType": "StockMovementRecorded",
  "aggregateId": "mov_01HZ_INSTALL_HG8245_WANJIKU",
  "timestamp": "2026-02-23T15:45:00.000Z",
  "payload": {
    "operatorCode": "WIK",
    "movementId": "mov_01HZ_INSTALL_HG8245_WANJIKU",
    "skuId": "WIK-ONT-HG8245-H5",
    "reasonCode": "INSTALL",
    "movementDirection": "OUTBOUND",
    "qty": 1,
    "sourceLocationId": "WIK-VAN-wik-internal-tech-team",
    "targetLocationId": null,
    "woId": "wo_01HZ_WANJIKU_INSTALL",
    "consumedReservations": ["rsv_01HZ_WANJIKU_HG8245"],
    "balanceAfter": {
      "skuId": "WIK-ONT-HG8245-H5",
      "locationId": "WIK-VAN-wik-internal-tech-team",
      "qtyOnHand": 4,
      "qtyReserved": 0,
      "qtyAvailable": 4
    },
    "createdBy": "service-account-wo-01"
  }
}
```

BIL-01 consumes this event to trigger the equipment deposit charge for Hon. Wanjiku's account (using the historical deposit amount from PLM-CFG-06 active at the install time, per PLM-CFG-06 R-PLM-EQ-4).

6.2 Sample event — StockLowAlert

```
{
  "eventType": "StockLowAlert",
  "aggregateId": "WIK-VAN-contractor_ke_nairobi_k",
  "timestamp": "2026-05-20T10:15:00.000Z",
  "payload": {
    "operatorCode": "WIK",
    "locationId": "WIK-VAN-contractor_ke_nairobi_k",
    "contractorId": "contractor_ke_nairobi_k",
    "skuId": "WIK-RTR-HAP-AC2",
    "qtyOnHand": 2,
    "qtyReserved": 2,
    "qtyAvailable": 0,
    "thresholdConfigured": 5,
    "lastReplenishmentAt": "2026-04-12T08:30:00Z",
    "averageDailyConsumption": 1.2,
    "estimatedStockoutAt": "now",
    "actionSuggested": "Schedule replenishment transfer from WIK-CENTRAL-NRB"
  }
}
```



```
}
}
```

Ops dashboards surface this prominently; the EM-02 contractor performance feed records it.

7. Stub retirement + future expansion

OSR-01 v1.0 does not retire any existing stub from another DD. It DOES create one stub of its own and lays the foundation for two future DDs that will retire OSR-01-side scope.

7.1 Stub created by this DD

`osr_contractor_stub(contractor_id PK, contractor_name)` — a minimal placeholder satisfying the FK reference from `stock_location.contractor_id` (for `CONTRACTOR_VAN` rows). Retired when EM-02 (Contractor & Staff Registry) lands. The cutover pattern follows the standard: deploy EM-02; backfill contractor identities to EM-02's contractor table; switch the FK in `stock_location.contractor_id` to point at EM-02's contractor PK; drop `osr_contractor_stub`.

7.2 Future OSR-INSTANCE-01 (or equivalent)

When individual equipment instance tracking is needed (likely driven by RMA workflow requirements), a successor DD will add:

- `equipment_instance(instance_id PK, sku_id FK, serial_number, current_location_id, current_status)` — one row per physical unit.
- `equipment_instance_lifecycle_event` — append-only state transitions per instance.
- A migration path: OSR-01's SKU+qty movement records are bulk-extended with synthetic `instance_id` references where reconstructable, or marked as unverifiable pre-OSR-INSTANCE-01.

7.3 Future OSR-RMA-01 (or equivalent)

RMA / repair / refurbishment workflows are out of v1.0 scope. The future DD will:

- Orchestrate the vendor-RMA process: write-off-pending → returned-to-vendor → repaired → received-back.
- Provide RMA tracking at the instance level (depends on OSR-INSTANCE-01).
- Integrate with WO module for return-WO types.

8. Cross-DD coordination

8.1 Upstream callers

Caller	Endpoint	When
WO module FRAMEWORK §9 (FINALIZATION)	POST <code>/api/stock-availability</code>	Before finalizing WO — ensure tech's van has the equipment
WO module FLOW-INSTALLATION	POST <code>/api/stock-reservations</code>	At WO finalization — reserve the qty against the van
WO module FLOW-INSTALLATION	POST <code>/api/stock-movements</code> (reason=INSTALL)	When tech reports install completion
WO module FLOW-DISCONNECT	POST <code>/api/stock-movements</code> (reason=RETURN)	When tech reports equipment recovered at disconnect
WO module (any flow)	DELETE <code>/api/stock-reservations/{id}</code>	WO cancellation
Procurement (pending DD)	POST <code>/api/stock-movements</code> (reason=VENDOR_RECEIPT)	At vendor delivery
BO admin UI / ops	POST <code>/api/stock-transfers</code> , POST <code>/api/stock-cycle-counts</code>	Replenishment and audits
EM-02	POST <code>/api/stock-locations</code>	Contractor onboarding (create the van location)

8.2 Downstream consumers (Kafka)

Consumer	Events subscribed	Action
BIL-01 Charging Engine	StockMovementRecorded (reason=INSTALL)	Apply equipment deposit + first month's rental to customer account
BIL-01 Charging Engine	StockMovementRecorded (reason=RETURN)	Trigger deposit refund adjustment (PLM-CFG-05 ADJ-EQUIPMENT-DEPOSIT-RETURN)
WO module	StockReservationExpired, StockReservationConsumed	Update WO status; flag for review on expiry
EM-02 performance metrics	StockMovementRecorded (filtered by reason + contractor)	Compute contractor on-time-delivery, write-off rates
ILM-CFG-01 customer account flag system	StockMovementRecorded (reason=RETURN)	Clear "equipment-not-returned" flag once return is recorded
Ops dashboards	All <code>sophix.osr.sc.*</code>	Stock topology, low alerts, cycle count results

8.3 Sibling modules

Module	Direction	Contract
PLM-CFG-06 (Equipment Catalog)	OSR-01 → PLM-CFG-06 (read)	OSR-01 references <code>equipment_sku.sku_id</code> as FK on every row. PLM-CFG-06 is upstream; SKU additions/deactivations propagate.
EM-02 (Contractor & Staff Registry)	OSR-01 → EM-02 (read; via stub in v1.0)	OSR-01 references <code>contractor_id</code> on <code>stock_location</code> for vans. v1.0 uses <code>osr_contractor_stub</code> ; retired on EM-02 landing.
WO module	OSR-01 ← WO (write reservations + movements) + OSR-01 → WO (via Kafka events)	Heaviest integration. WO finalization triggers reservation; WO install completion triggers INSTALL movement; WO cancellation releases reservation.
BIL-01	BIL-01 ← OSR-01 (via Kafka)	INSTALL movement events trigger equipment charges; RETURN movement events trigger deposit-refund adjustments.
ILM-CFG-01	ILM-CFG-01 ← OSR-01 (via Kafka)	RETURN movements at disconnect feed into customer-account flag system for "equipment-not-returned" handling.
Procurement DD (pending)	OSR-01 ← Procurement (write <code>VENDOR_RECEIPT</code> movements)	Procurement DD records the vendor PO + delivery; calls OSR-01 to record the stock receipt movement.

8.4 Dependencies (build order)

- PLM-CFG-06** (Equipment Catalog) — already shipped; provides `equipment_sku.sku_id` references.
- `osr_contractor_stub` — minimal table seeded with contractor IDs from ILM-CFG-02 sample data + operator onboarding. v1.0 ships with seed for KE WIK + YASSN + WUG + WTZ.
- Stock location seed** — per-operator seed scripts create the `CENTRAL_WAREHOUSE` row + one `CONTRACTOR_VAN` row per active contractor.
- Reason code seed** — per-operator seed populates the standard reason codes (`VENDOR_RECEIPT`, `TRANSFER_*`, `INSTALL`, `RETURN`, `WRITE_OFF_*`, `CYCLE_COUNT_*`).
- Initial stock counts** — for migration from any pre-v1.0 inventory system, a one-time job ingests opening balances as `VENDOR_RECEIPT` movements dated `effective_from` of the operator's go-live.
- WO module integration** — WO finalization, install completion, cancellation, and disconnect flows are updated to call OSR-01 reservation + movement APIs.
- EM-02 (shipped — retirement plan in §7)** — OSR-01 v1.0 ships and operates with `osr_contractor_stub`. EM-02 landing later swaps the FK target.

9. Workshop sources

Workshop deep-dive sources (Inventory deep-dive) present in the project but PDFs unreadable for text extraction.

Stock chain functional requirements are referenced implicitly across:

- **Inventory workshop** (dedicated deep-dive) — stock topology, central-vs-van, replenishment, cycle counts.
- **Work Order workshop** (deep-dive 07) — install + return flows that touch stock at the van.

- **Provisioning workshop** (deep-dive 05) — equipment-on-customer-premises post-install.

V3 design intent: **two-tier stock topology with immutable movement ledger** — provides full audit trail for stock balance changes while keeping the model simple enough to operate (no per-instance tracking complexity in v1.0). The TZ codebase's current handling of stock is unverified pending cartography; placeholder: "TBD pending codebase review."

Out of v1.0 scope:

- **Individual equipment instance / serial number tracking** — deferred to future OSR-INSTANCE-01 (or equivalent). v1.0 works at (sku_id, location_id, qty) level only.
- **RMA / repair / refurbishment workflows** — deferred to future OSR-RMA-01.
- **Van-to-van transfers** — v1.0 forces routing through central. Future enhancement may relax.
- **Customer premises as a tracked location** — v1.0 records install as "left the van"; the customer side is opaque. Will become natural once instance tracking lands.
- **Vendor procurement orchestration** — PO management, vendor SLAs, lead times — out (separate Procurement DD).
- **Multi-currency stock value reporting** — out; v1.0 tracks qty only, no inline cost basis. Stock valuation can be derived externally from movement + equipment fee data.
- **Operator-configurable cycle count cadence + auto-scheduling** — out; v1.0 supports ad-hoc cycle count submission via API only.

End of DD_OSR-01-Stock_Chain_Engine-v1.0.